Subject: Designing a Centralized Login Portal with Zero Trust, OAuth2, and MFA

Course: Secure Software Development

Students : Nicklas Kramer & Anders Hoffmann

First Run

1. Database. The app won't run until the database is set up. Apply the migrations with:

```
dotnet ef database update
```

2. Keycloak (for OAuth2/OIDC login). Start Keycloak:

```
docker compose up -d
```

Then open the admin console at `http://localhost:8080` (login `admin` / `admin`) and:

1. Create a realm named `loginportal`.
2. In that realm, create a client with Client ID `loginportal-backend`, set **Client authentication** to On (confidential), and add the redirect URI `http://localhost:5256/signin-oidc`.
3. Copy the client's secret (Credentials tab) into `Oidc:ClientSecret` in `src/LoginPortal.Backend/appsettings.json`.

Add at least one user to the realm so you have something to log in with.

1. Introduction and problem

This project is a centralized login portal that acts as a single entry point to backend resources, securing the user journey from authentication to authorization. It combines OAuth2/OpenID Connect for SSO, JWT tokens for sessions, MFA/2FA, and RBAC, all built on a Zero Trust principle ("never trust, always verify").

Problem: How can a centralized login portal be designed and implemented so that it protects against identity-based attacks (such as session hijacking) by combining Zero Trust architecture

with modern protocols like OAuth2 and MFA?

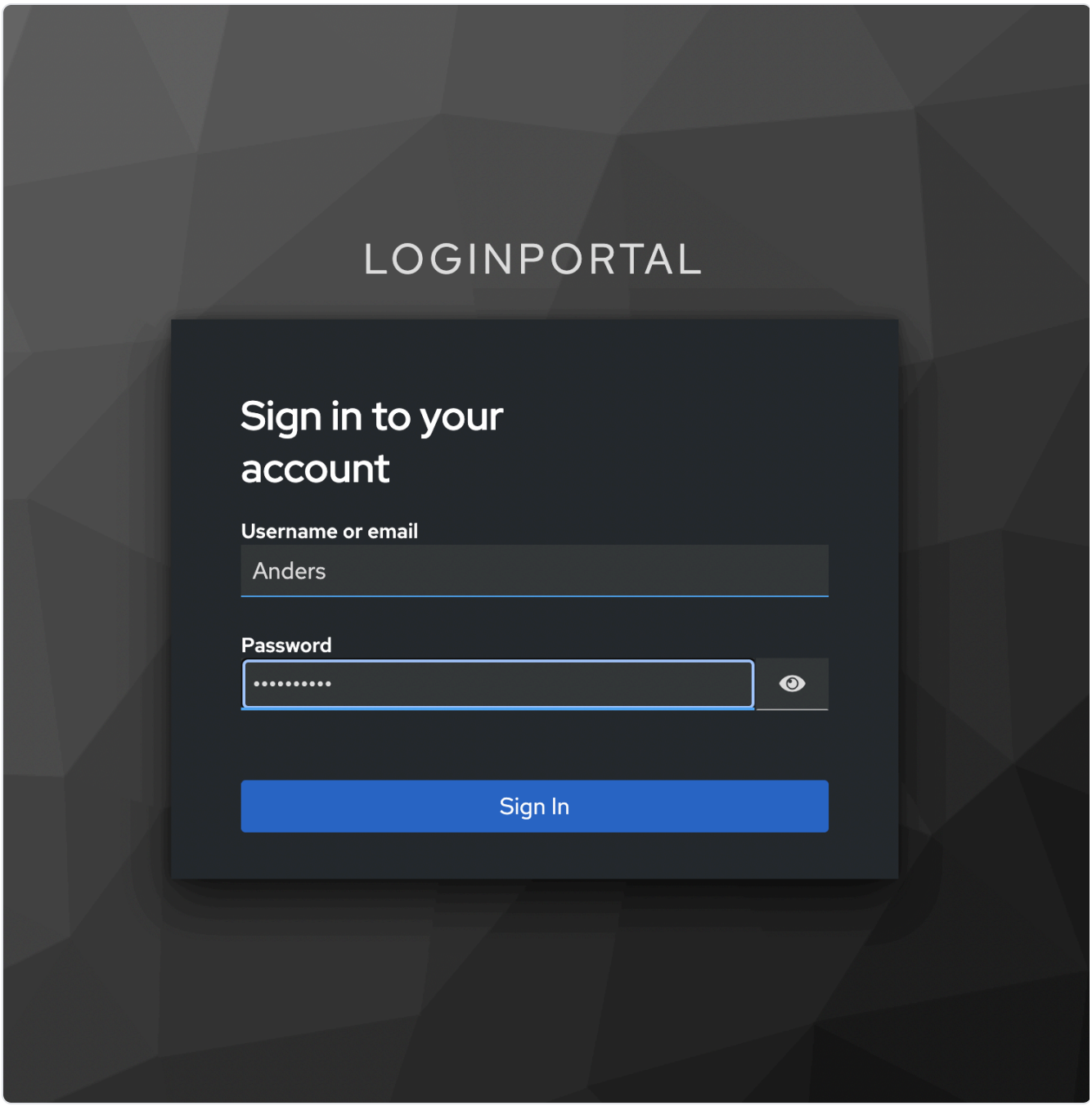
2. Practical

2.1 Federated authentication: OAuth2 and OpenID Connect

Users can log in through **Keycloak** instead of the portal's own login. We use OpenID Connect with the OAuth2 Authorization Code flow and PKCE, asking for the `openid`, `profile` and `email` scopes.

The user clicks "Login With KeyCloak", we redirect them to Keycloak to sign in, and Keycloak redirects back to our callback. The first time, we create a local `IdentityUser` for them and give them the `User` role. After that the backend hands out its own JWT, exactly like a normal login, so Zero Trust and RBAC work the same no matter how the user signed in.

It's implemented with `AddOpenIdConnect` in `LoginPortal.Backend/Program.cs`, the redirect and callback are `ExternalLogin` / `ExternalCallback` in `LoginPortal.Backend/Controllers/AuthController.cs`, the button is in `LoginPortal/Pages/Login.cshtml`, and the JWT cookie is set in `LoginPortal/Pages/Account/External.cshtml.cs`. The Keycloak login screen:



2.2 Cryptographic primitives: password hashing and JWT signing

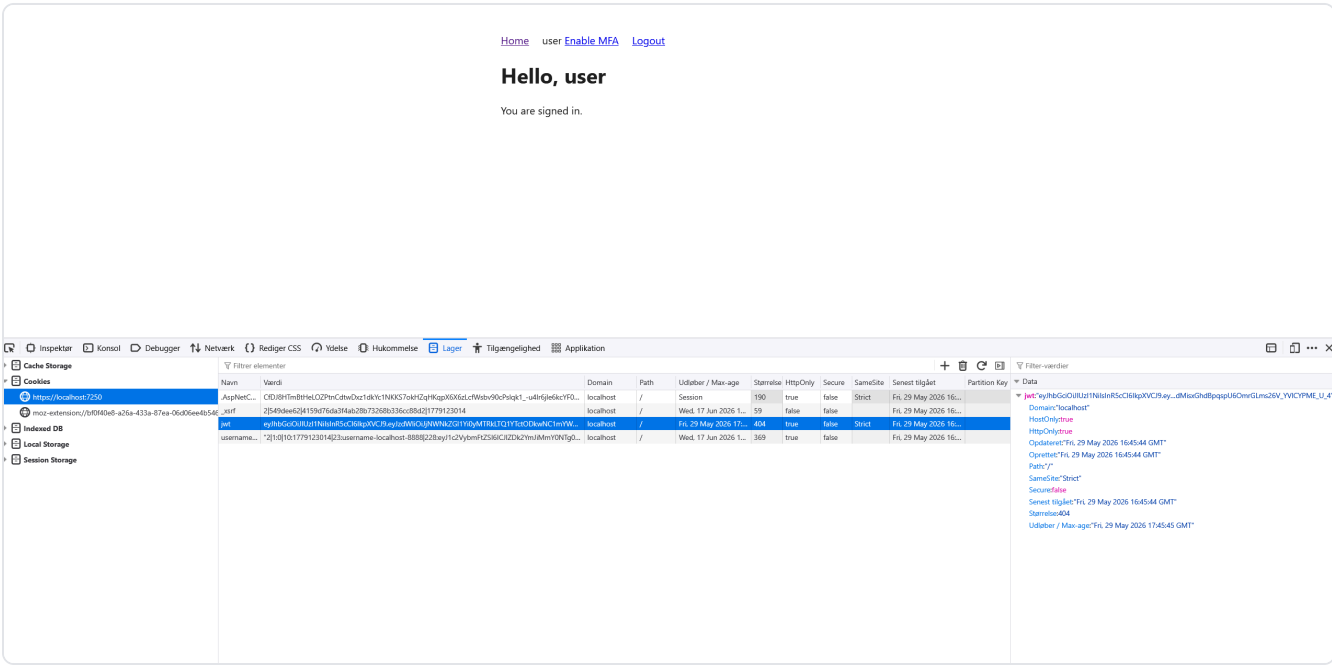
This project uses EF Core identity to manage users. EF Core Identity automatically hashses any passwords using the PBKDF2 algorithm with the HMAC-SHA256 function and uses a 128 bit random salt. This way no passwords are ever stored in plain text, and password sent through HTTPS are encrypted.

Any authenticated user gets sent a JWT token with claims through the HTTPS respone to their browser, which is stored as a cookie named "jwt". This cookie expires after 1 hour, prompting the

user to log in again.

For any required endpoints where the backend needs to authorize the caller, the JWT is sent in the header as a bearer token. The backend then validates it and returns the relevant response.

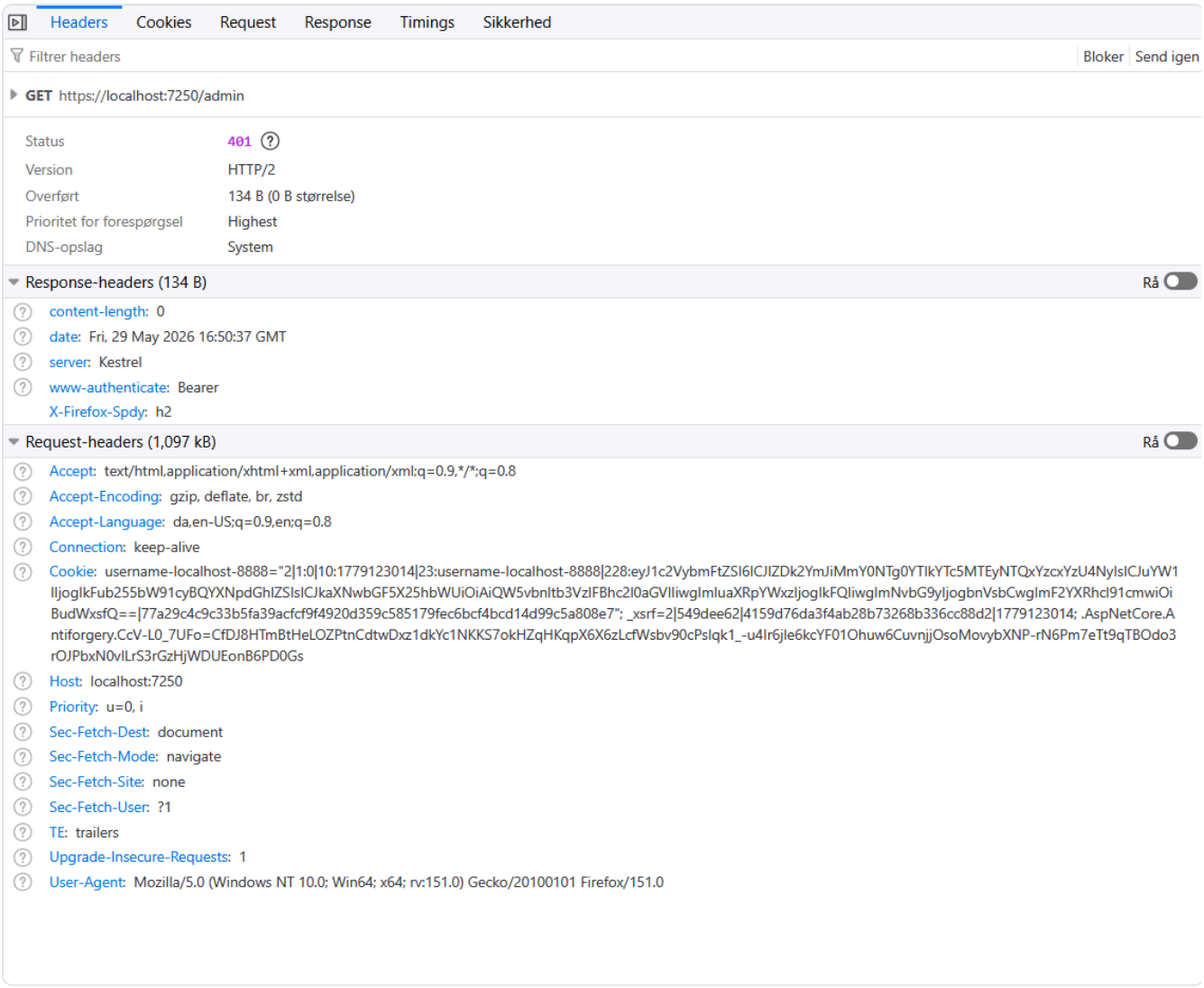
In code, password hashing is handled by `AddIdentity<IdentityUser, IdentityRole>()` in `LoginPortal.Backend/Program.cs`, while JWT creation and HMAC-SHA256 signing live in `LoginPortal.Backend/Services/JwtService.cs`. Below is a screenshot of the issued JWT:



2.3 Zero Trust architecture

The project follows the "never trust, always verify" principle: no request is implicitly trusted, not even between our own frontend and backend. Every call into the backend is re-authenticated by validating the JWT's signature, issuer, audience and expiry on each request, and every protected endpoint runs its own `[Authorize]` check rather than relying on a shared session.

In code this is set up by `AddJwtBearer` with `TokenValidationParameters` in `LoginPortal.Backend/Program.cs`, and the JWT cookie issued in `LoginPortal/Pages/Login.cshtml.cs` is `HttpOnly` + `SameSite=Strict` with a 1 hour lifetime so it cannot be read by JavaScript and is short-lived even if leaked. The screenshot below shows a request being rejected when the JWT is missing or invalid:



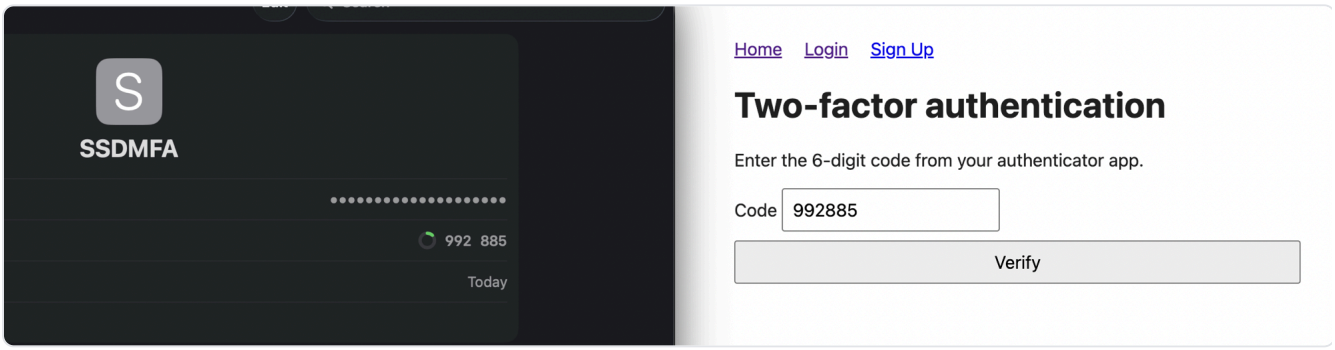
2.4 Multi-Factor Authentication (MFA/2FA)

Users can add a second factor with an authenticator app (Apple Keychain, Microsoft Authenticator, etc.). We use ASP.NET Core Identity's built-in TOTP support, and each user's secret is stored in the `AspNetUserTokens` table and only shown once during setup.

To turn it on, the user opens the MFA setup page, scans the QR code/secret into their app, and types back a 6-digit code to confirm. After that, entering the right password isn't enough on its own: the backend gives back a short-lived token, and the user has to enter a valid 6-digit code before they get the normal 1 hour JWT.

It's implemented with the setup endpoints in `LoginPortal.Backend/Controllers/MfaController.cs`, the login check and verify step are in `LoginPortal.Backend/Controllers/AuthController.cs`, and the pending token is handled by `GenerateMfaPendingToken` / `ValidateMfaPendingToken` in

`LoginPortal.Backend/Services/JwtService.cs`. The pages are `LoginPortal/Pages/Account/MfaSetup.cshtml` (turn on) and `LoginPortal/Pages/Account/Mfa.cshtml` (enter code at login). The code prompt at login, next to the matching code in the authenticator app:

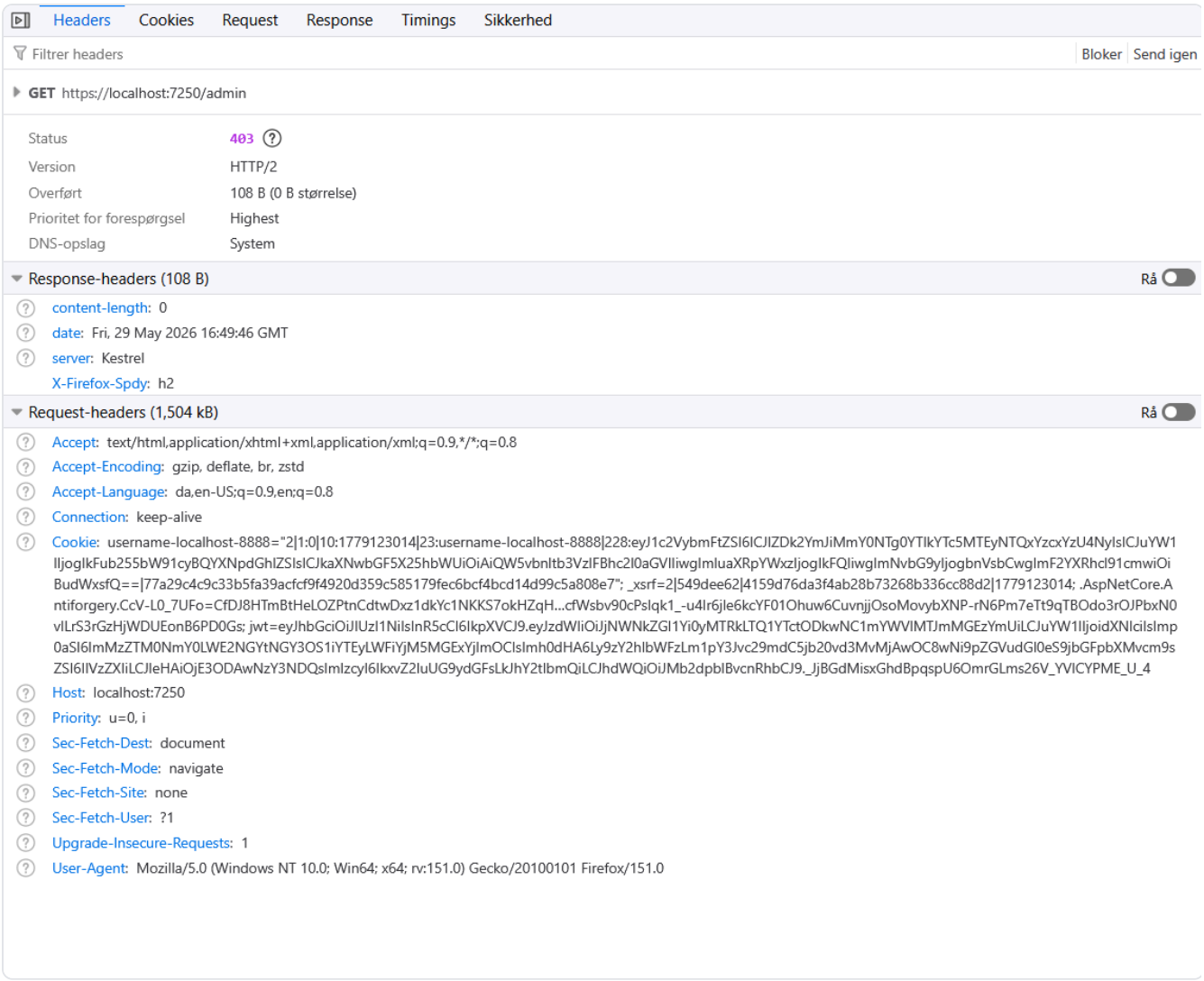


2.5 Role-Based Access Control (RBAC)

This project is using Role-Based Access Control to dictate which roles has access to certain areas of the system. In this project this is done using claims that exist within the JWT, which the frontend can read and see which role that the authenticated user has.

Any requests to the backend are also being role checked and authorizing whether or not the user has permission to access the attempted endpoint. Unauthorized users will recieve a 401 Unauthorized response while unauthenticated users will recieve a 403 Forbidden response.

In practice this is implemented through ASP.NET Core's `[Authorize(Roles = "Admin")]` attribute on protected endpoints (see `LoginPortal.Backend/Controllers/AuthController.cs` and the `Admin` Razor page), with the `Admin` and `User` roles seeded in `LoginPortal.Backend/Data/SeedData.cs`. The screenshot below shows a non-admin user being blocked from the admin area:



2.6 Countermeasures against session hijacking

To make a stolen session as hard and as useless as possible, several layers are combined: the JWT cookie is marked `HttpOnly` so JavaScript (and therefore XSS payloads) cannot read it, `SameSite=Strict` so it is not sent on cross-site requests, and given a short 1 hour lifetime so a leaked token expires quickly. The token itself is signed with HMAC-SHA256, so an attacker cannot tamper with claims, and MFA adds an extra factor that a stolen password alone cannot bypass.

In code the cookie flags are set in `LoginPortal/Pages/Login.cshtml.cs` (`SetJwtCookie`), and signature/lifetime validation happens on every request via `AddJwtBearer` in `LoginPortal.Backend/Program.cs`.